MIPS 5 段流水线模拟器设计与验证

姓名：李祥熙　学号：2234412799

2026 年 5 月 15 日

1　实验目的

本实验的目标如下：

- 加深对计算机流水线基本概念的理解
- 理解 MIPS 结构如何用 5 段流水线来实现，理解各段的功能和基本操作
- 加深对数据冲突、结构冲突、控制冲突的理解，并能够分析这些冲突对 CPU 性能的影响
- 进一步理解解决数据冲突的方法，掌握如何应用定性技术来减少数据冲突引起的流水线停顿

2　实验过程

2.1　实验环境

- 操作系统：Windows 11
- 编程语言：Python 3

2.2　实现步骤

1. 设计 5 段流水线寄存器结构（IF/ID, ID/EX, EX/MEM, MEM/WB）并明确每段职责
2. 实现指令解析、标签解析与数据段内存初始化，建立指令索引与地址对应关系
3. 实现数据前递、load-use 冒险检测、分支冲刷与终止指令处理
4. 构建命令行交互：单步执行、断点运行、流水线与寄存器查看、性能统计
5. 使用三类示例程序验证无冒险、RAW 冒险与分支控制冒险

3 实验内容

3.1 任务一：模拟器设计原则与特性

实现思路与总体流程：

- **指令与数据的两遍解析：**第一次扫描.text/.data，记录代码段标签的指令索引和数据段标签的字节地址；第二次扫描将.text 指令转为内部结构，同时把.data 的.word 写入模拟内存。
- **5 段流水线状态建模：**为 IF/ID、ID/EX、EX/MEM、MEM/WB 分别建立寄存器结构，保存指令与必要的中间结果（如寄存器值、ALU 结果、访存控制信号）。
- **逐周期推进：**每个时钟周期按 WB→MEM→EX→ID→IF 顺序更新下一周期的流水线寄存器，保证写回先于读取，体现真实数据流。
- **冲突检测与处理：**在 ID 阶段检测 load-use 冒险；若发现，插入气泡并冻结 IF/ID。EX 阶段支持转发，从 EX/MEM 或 MEM/WB 直接取值，减少 RAW 引起的停顿。
- **控制冒险处理：**分支在 EX 阶段判定，命中后冲刷 IF/ID 与 ID/EX；以 TEQ 作为软件终止指令，触发管线清空并结束执行。
- **交互式命令行：**提供 step/run/break/list/pipe/regs/stats 等命令，既能观察每周期流水线状态，也能统计性能指标。

关键实现代码与功能说明：

- **两遍解析与数据段写入：**先建立代码与数据标签，再把.word 写入模拟内存。

```
def _first_pass(self, lines):
    section = "text"; pc = 0; data_addr = 0
    for raw in lines:
        line = raw.split("#", 1)[0].strip()
        if line.lower() == ".text": section = "text"; continue
        if line.lower() == ".data": section = "data"; continue
        if ":" in line:
            label, rest = [p.strip() for p in line.split(":", 1)]
            if section == "text": self.text_labels[label] = pc
            else: self.data_labels[label] = data_addr
            line = rest
        if section == "text": pc += 1
        else: data_addr += 4 * len(self._split_args(line[5:]))

def _parse_instructions(self, lines):
    section = "text"; data_addr = 0
```

```
    for raw in lines:
        line = raw.split("#", 1)[0].strip()
        if line.lower() == ".data": section = "data"; continue
        if section == "data" and line.lower().startswith(".word"):
            for val in self._split_args(line[5:]):
                self._write_mem_word(data_addr, self._parse_imm(val
                    ))
                data_addr += 4
```

- **逐周期推进与 5 段流水线更新：**采用“旧值读、新值写”的方式构建下一拍流水线寄存器，确保时序正确。

```
def step(self):
    old_ifid, old_idex = self.ifid, self.idex
    old_exmem, old_memwb = self.exmem, self.memwb

    # WB
    if not old_memwb.instr.is_nop():
        if old_memwb.reg_write:
            self._reg_write(old_memwb.dest, wb_val)
        self.completed += 1

    # MEM -> new_memwb, EX -> new_exmem, ID -> new_idex, IF ->
       new_ifid
    self.ifid, self.idex = new_ifid, new_idex
    self.exmem, self.memwb = new_exmem, new_memwb
```

- **load-use 冒险检测：** ID 阶段检查 EX 阶段是否为 LW，若使用同一寄存器则插入气泡。

```
def _should_stall(self, ifid, idex, exmem, memwb):
    rs, rt = self._get_source_regs(ifid.instr)
    if idex.instr.op == "lw":
        load_rt = idex.instr.args[0]
        if load_rt != 0 and (load_rt == rs or load_rt == rt):
            return True
    return False
```

- **转发实现：**当 EX/MEM 或 MEM/WB 将写回时，直接覆盖 EX 阶段操作数，减少 RAW 停顿。

```
def _apply_forwarding(self, idex):
```

```
    rs_val, rt_val = idex.rs_val, idex.rt_val
    if self.exmem.reg_write and self.exmem.dest != 0:
        if self.exmem.dest == idex.rs: rs_val = self.exmem.
            alu_result
        if self.exmem.dest == idex.rt: rt_val = self.exmem.
            alu_result
    if self.memwb.reg_write and self.memwb.dest != 0:
        wb_val = self.memwb.mem_data if self.memwb.mem_to_reg else
            self.memwb.alu_result
        if self.memwb.dest == idex.rs: rs_val = wb_val
        if self.memwb.dest == idex.rt: rt_val = wb_val
    return rs_val, rt_val
```

- **控制冒险与冲刷：**分支在 EX 阶段判定，命中后清空前段并更新 PC。

```
if branch_taken:
    next_pc = branch_target
    new_ifid = IFID(Instruction.nop(), next_pc)
    new_idex = IDEX(Instruction.nop(), 0, 0, 0, 0, 0, 0, 0)
```

- **交互功能示例：**源码列表命令展示指令地址与当前 PC 位置，便于演示。

```
def dump_source(self, start=0, count=0):
    for i in range(start, end):
        addr = i * 4
        marker = "->" if i == self.pc else "  "
        lines.append(f"{marker} {i:04d} (0x{addr:08x}): {self.
            program[i].text}")
```

设计原则：

- 与课堂 5 段流水线一致：IF 取指、ID 译码、EX 执行、MEM 访存、WB 回写
- 与指令级行为对齐：支持 `ADDIU/ADD/LW/SW/BEQ/BEQZ/TEQ`，并以 `TEQ` 作为程序终止指令
- 冒险处理可观察：支持转发开关，遇到 load-use 时插入气泡
- 可交互：支持单步、断点、运行到指定 PC，随时查看流水线寄存器、通用寄存器与内存

关键特性：

- 源码列表功能：以指令索引和字节地址显示源码，并用箭头标记当前 PC

- 性能统计：输出周期数、完成指令数、停顿与冲刷次数，计算 CPI
- 控制冒险处理：分支在 EX 段判定，发生跳转时冲刷流水线前段

3.2 任务二：没有任何冲突的流水线场景示例（no_hazard.s）

目的：验证无明显数据冒险时流水线能够连续填充并高效执行。

代码：

```
.text
main:
ADDIU $r8,$r0,A
ADDIU $r9,$r0,B
ADDIU $r10,$r0,C
ADDIU $r11,$r0,RES1
ADDIU $r12,$r0,RES2
LW    $r1,0($r8)
LW    $r2,0($r9)
LW    $r3,0($r10)
ADD   $r7,$r0,$r0
ADD   $r4,$r1,$r2
ADD   $r5,$r2,$r3
ADD   $r7,$r0,$r0
SW    $r4,0($r11)
SW    $r5,0($r12)
TEQ   $r0,$r0

.data
A:    .word 7
B:    .word 11
C:    .word 5
RES1: .word 0
RES2: .word 0
```

说明：前半段将数据段标签地址装入寄存器，随后依次加载 A/B/C，计算两组和并写回 RES1/RES2。整段程序数据相关性较弱，load 后有空隙指令，可体现流水线在无冒险情况下的连续执行特性。

3.3 任务三：有至少一次的 RAW 冲突场景示例（raw_hazard.s）

目的：通过“load 后立即使用”制造 RAW 冒险，观察转发与停顿对性能的影响。

代码：

```
.text
main:
ADDIU $r8,$r0,INC
ADDIU $r9,$r0,VAL
ADDIU $r10,$r0,OUT
LW    $r3,0($r8)
LW    $r1,0($r9)
ADD   $r2,$r1,$r3
SW    $r2,0($r10)
TEQ   $r0,$r0

.data
VAL:  .word 9
INC:  .word 1
OUT:  .word 0
```

说明：两条 LW 后紧接 ADD，构成典型 load-use 冒险。在开启转发时仍需插入 1 个气泡以等待数据从 MEM 阶段可用；关闭转发时需要额外停顿，CPI 升高。

3.4 任务四：有至少一次的分支跳转场景示例（branch_jump.s）

目的：通过条件分支制造控制冒险，观察分支判定与冲刷的行为。

代码：

```
.text
main:
ADDIU $r8,$r0,FLAG
ADDIU $r9,$r0,X
ADDIU $r10,$r0,Y
ADDIU $r11,$r0,OUT
LW    $r1,0($r8)
LW    $r2,0($r9)
LW    $r3,0($r10)
BEQ   $r1,$r0,DO_ADD
ADD   $r4,$r0,$r0
SW    $r4,0($r11)
TEQ   $r0,$r0

DO_ADD:
ADD   $r4,$r2,$r3
SW    $r4,0($r11)
```

```
TEQ    $r0,$r0

.data
FLAG:  .word 0
X:     .word 12
Y:     .word 30
OUT:   .word 0
```

说明：当 FLAG=0 时分支成立，跳转到 DO_ADD 执行加法并写回。分支在 EX 阶段判定，若跳转则冲刷前端，产生控制冒险带来的周期损失。

4 实验结果

4.1 任务一结果：无冒险执行过程

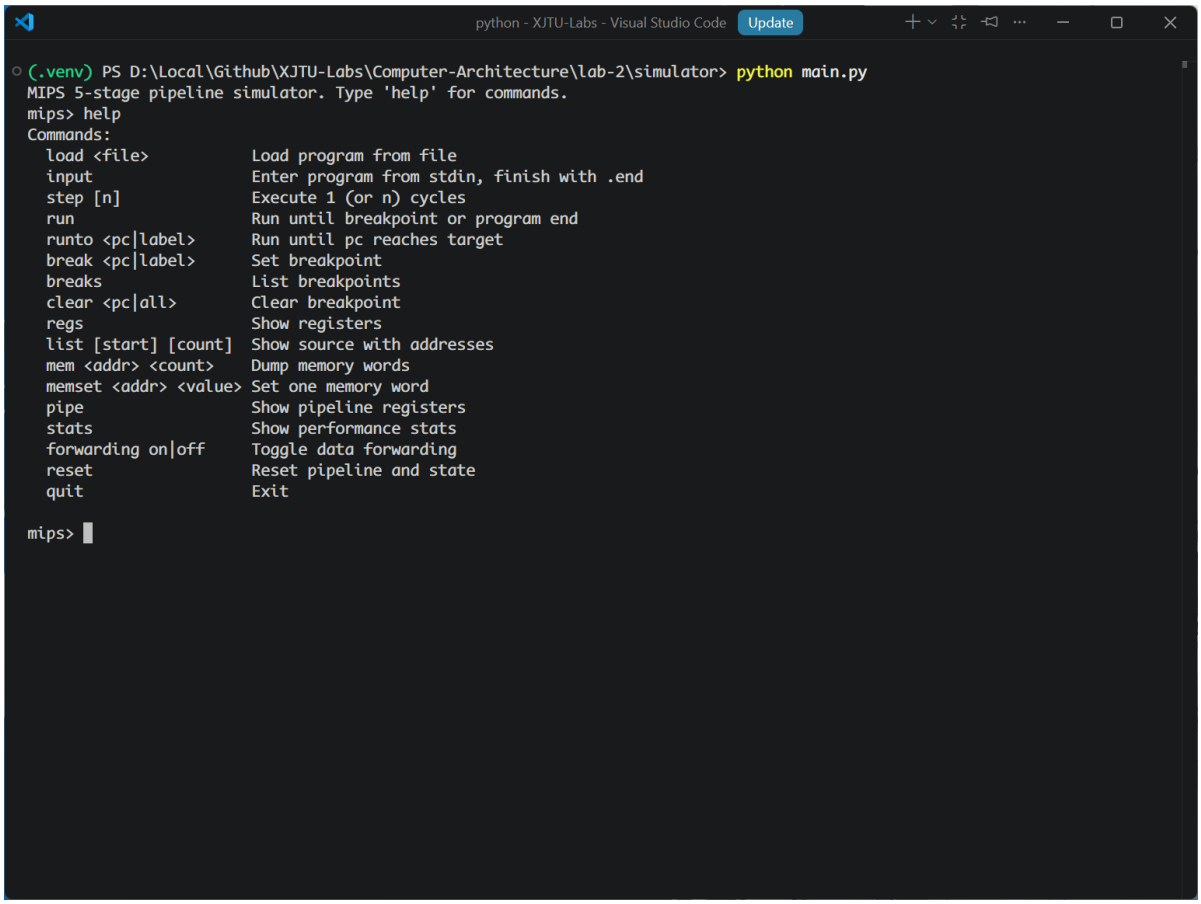

图 1: 命令行交互与功能提示

结果分析：图中展示模拟器命令行可用指令，包含源码列表、流水线查看、断点与统计等能力，为后续实验提供交互支持。

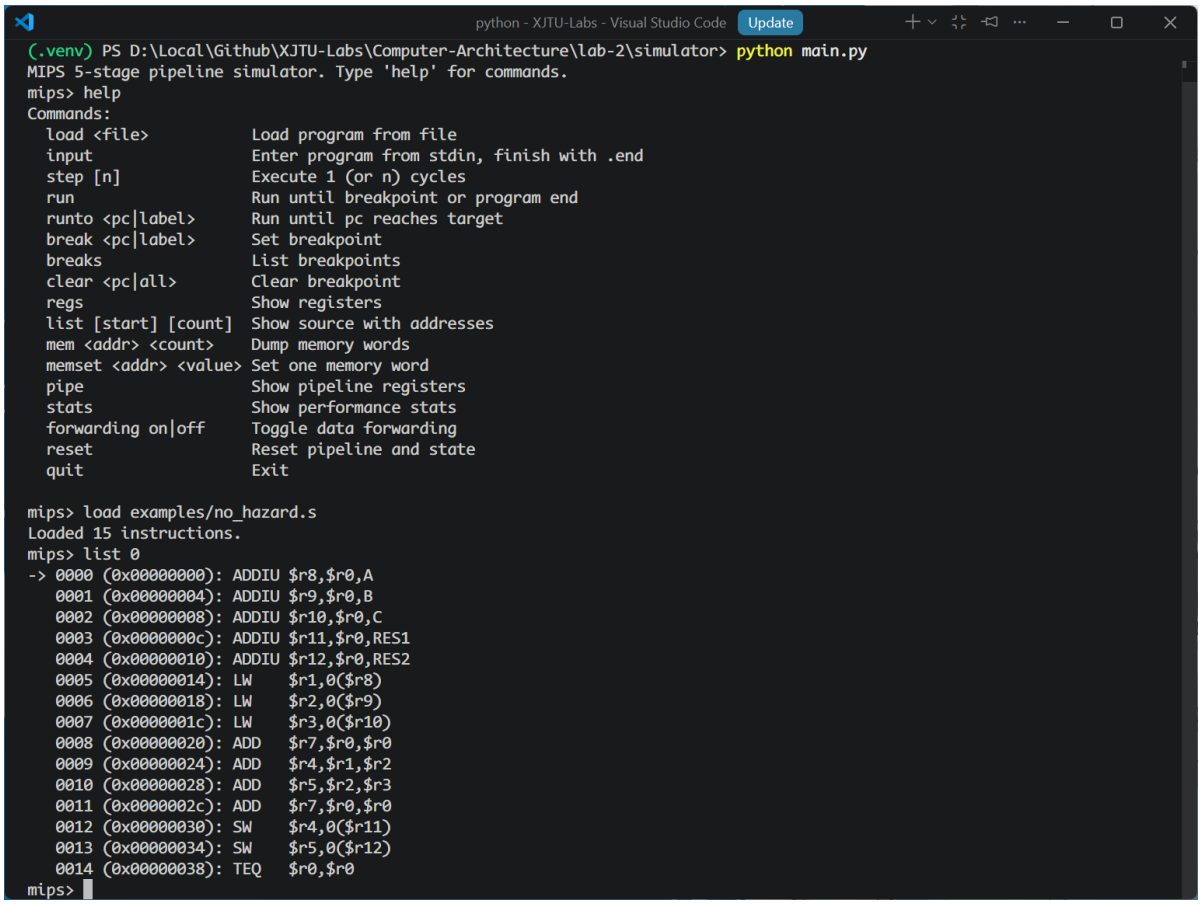

图 2: 加载无冒险程序并查看带地址源码

结果分析：源码以指令索引与字节地址展示，箭头指向当前 PC。可观察到程序从 ADDIU 开始顺序取指。该界面体现了“源码列表（list）+ PC 指示”的功能，可用于定位当前将要进入 IF 阶段的指令。当单步推进时，箭头随 PC 移动，能够直观看到取指顺序与分支跳转位置。

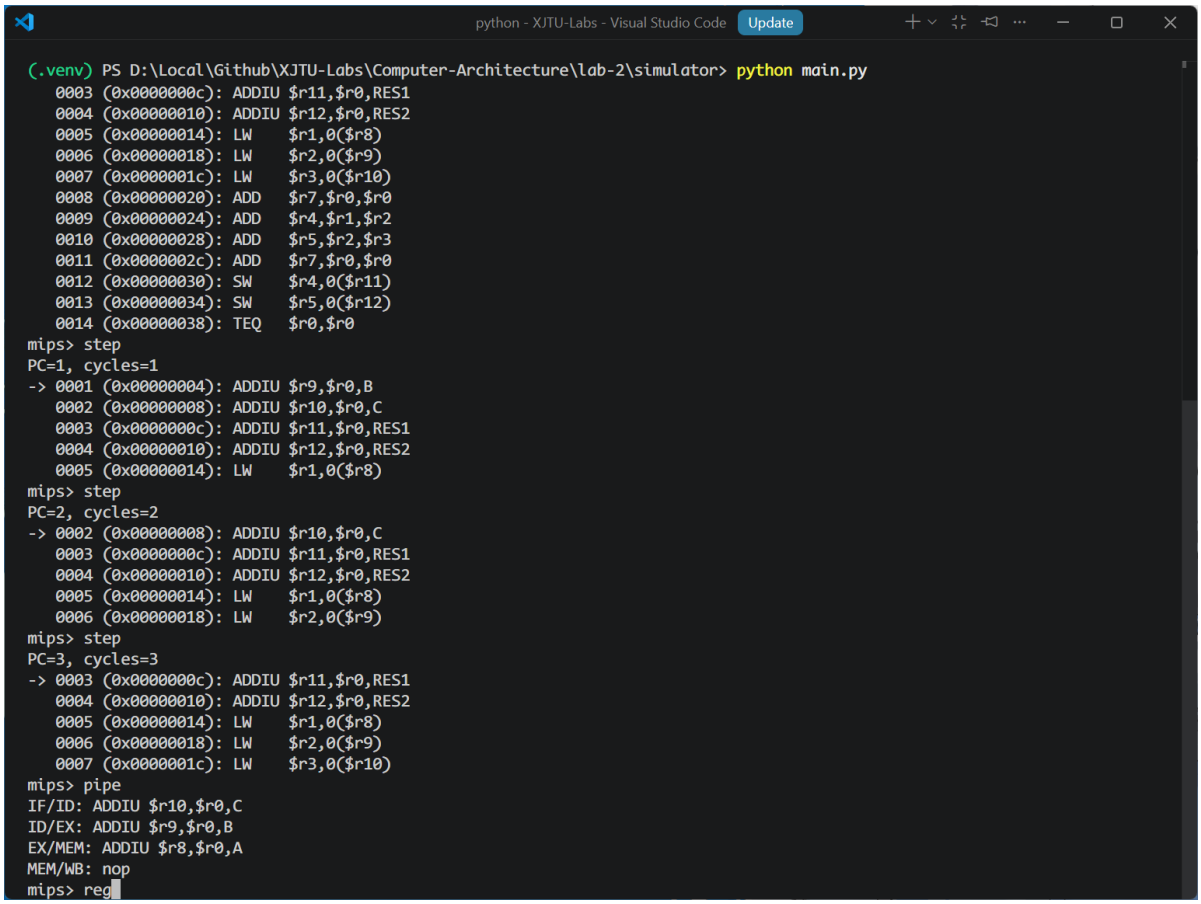

图 3: 单步执行与流水线寄存器状态

结果分析：单步执行后可见流水线各段指令随周期推进，IF/ID、ID/EX、EX/MEM、MEM/WB 依次填充。由于指令之间依赖少，流水线能够平稳流动，几乎不出现气泡。具体来看：前几拍 `ADDIU` 连续进入 IF 与 ID，随后进入 EX/MEM/WB；`LW` 在 EX 计算有效地址、MEM 读数据、WB 写回；由于中间存在与 `ADD` 的间隔，不会触发 load-use 停顿，流水线保持满载状态。

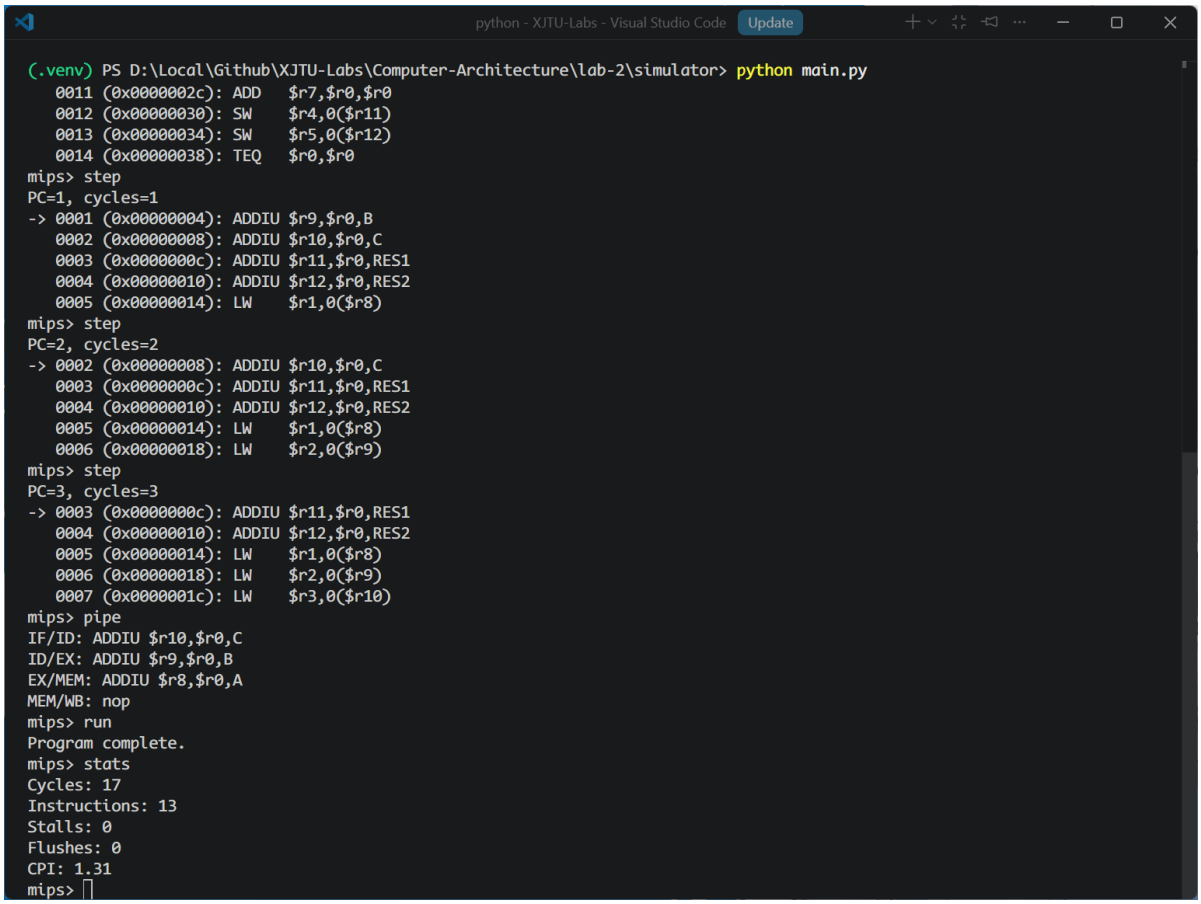

图 4: 无冒险程序运行完成与统计信息

结果分析：统计信息包含周期数、完成指令数、停顿与冲刷次数。CPI 通过

$$\mathrm{CPI} = \frac{\mathrm{Cycles}}{\mathrm{Instructions}} \tag{1}$$

计算，可看到无冒险情形下 CPI 接近 1。这说明大多数周期都用于有效指令执行，停顿与冲刷基本为零，符合“无冲突”样例的预期。

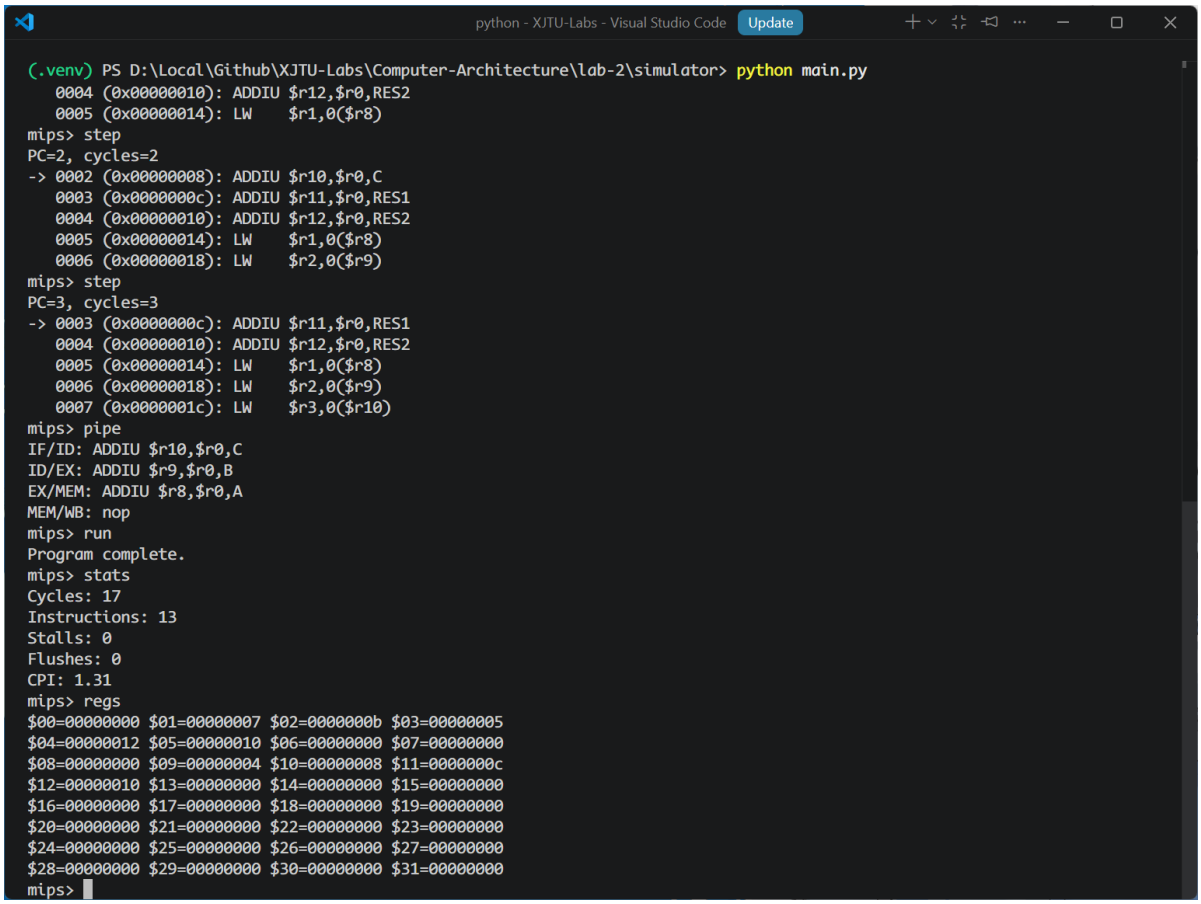

图 5: 无冒险程序结束后的寄存器状态

结果分析：寄存器中可观察到 RES1 与 RES2 对应的写回结果，验证计算与存储正确性。结合数据段初值 A=7，B=11，C=5，RES1=A+B、RES2=B+C 应分别为 18 与 16，寄存器与内存的最终值与之匹配，说明流水线执行与访存逻辑正确。

4.2 任务二结果：RAW 冒险与转发影响

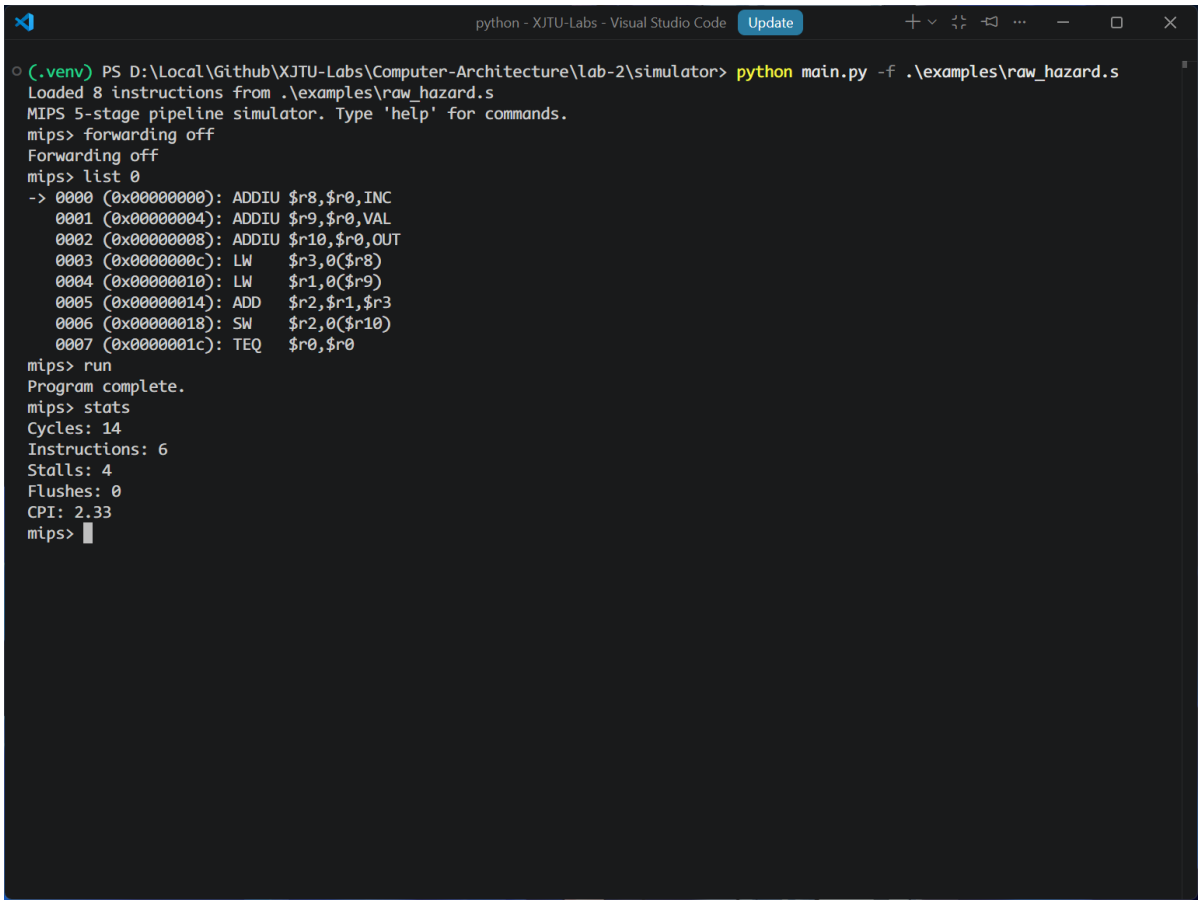

图 6: 关闭转发时的 RAW 冒险与统计

结果分析：LW 后紧随 ADD 导致 load-use 冒险。关闭转发时，流水线需要插入更多停顿，统计中停顿次数明显增加，CPI 上升。执行过程上，LW 在 MEM 阶段才产生有效数据，而 ADD 在 EX 阶段需要操作数，因此必须插入气泡等待写回完成；关闭转发后无法从中间寄存器取值，停顿次数进一步增加。该图反映了“转发开关”功能可直接影响性能统计，是模拟器的关键特性之一。

4.3 任务三结果：分支控制冒险与断点

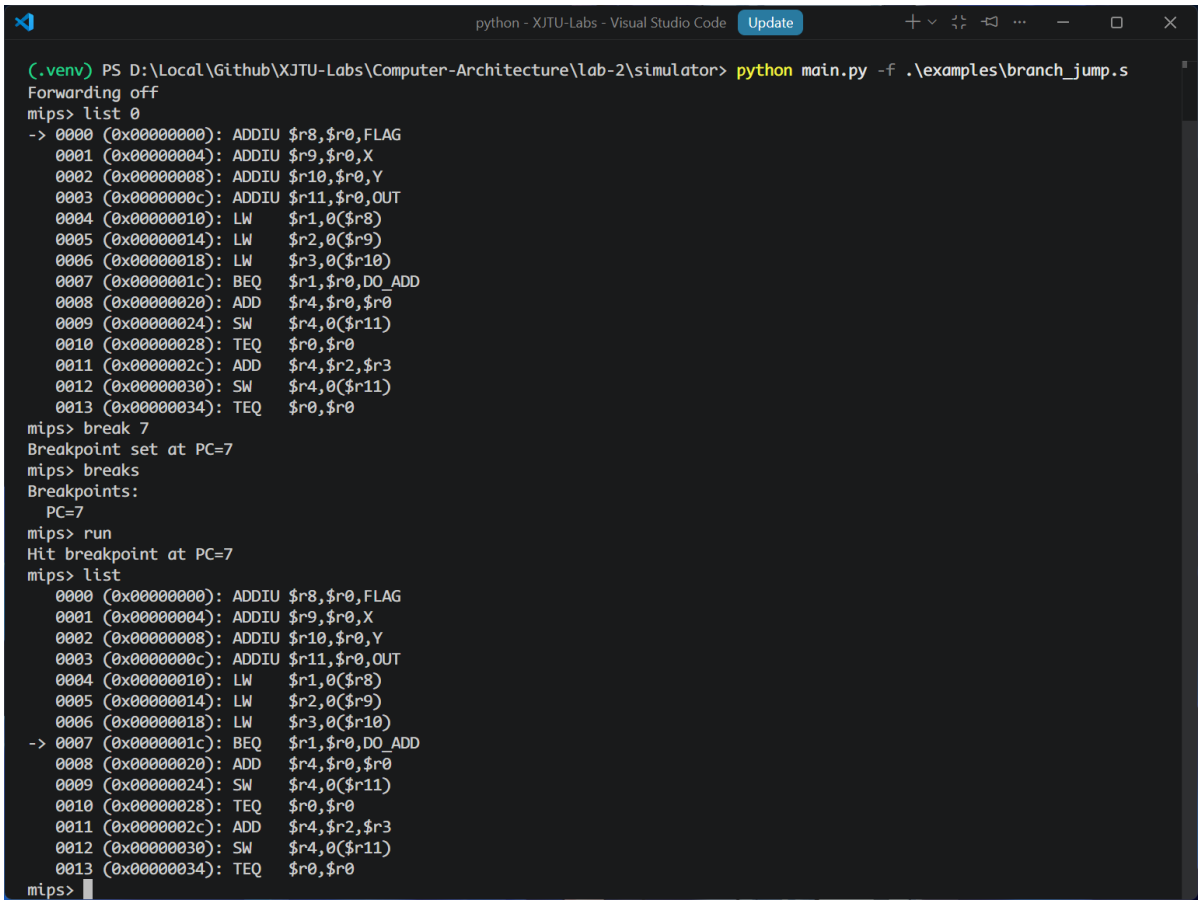

图 7: 分支程序的断点与执行流

结果分析：断点设置在分支目标附近，便于观察 BEQ 是否跳转。当 FLAG=0 时分支成立，PC 跳转到 DO_ADD，前端指令被冲刷。此处断点命中验证了 break 功能：运行过程中当 PC 到达指定指令索引时暂停，允许检查流水线与寄存器，定位控制冒险发生的具体时刻。从执行流看，BEQ 在 EX 阶段判定成立，IF/ID 与 ID/EX 中的顺序指令被冲刷，随后 PC 指向 DO_ADD 处开始取指，体现了控制冒险的处理策略。

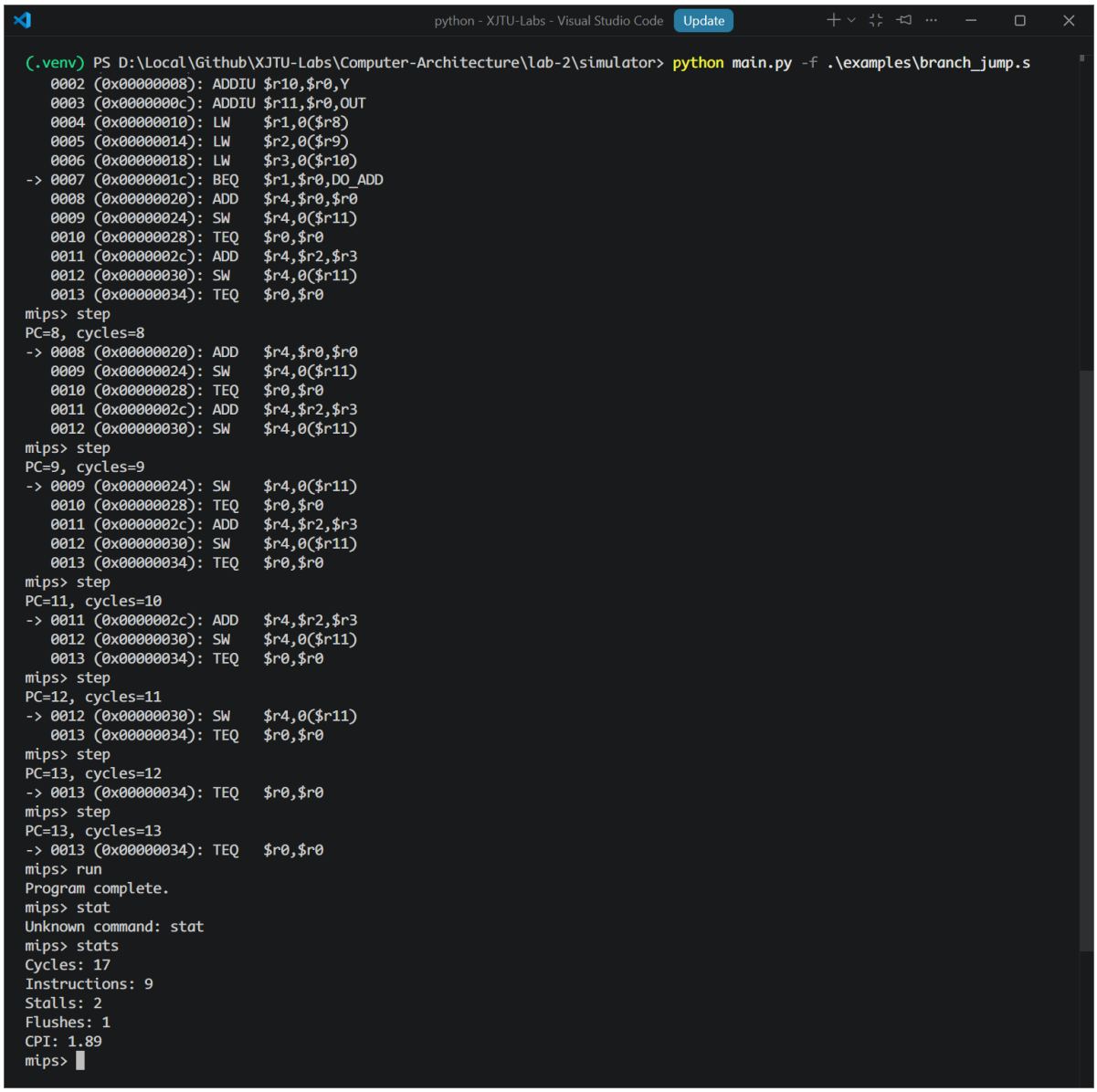

图 8: 分支程序运行完成与统计

结果分析：统计结果反映了控制冒险带来的冲刷开销，分支命中时冲刷次数增加。与无冒险程序相比，CPI 略高，体现控制冒险对性能的影响。该结果与断点图中的执行流一致：跳转成立会清空前段指令，导致额外周期，因此冲刷次数与 CPI 上升具有明确因果关系。

5 总结

本实验成功完成了 MIPS 5 段流水线模拟器的设计与验证：

- 实现了完整的 IF/ID/EX/MEM/WB 流水线，支持转发与冒险检测
- 使用无冒险、RAW 冒险与分支冒险三类程序验证执行正确性与性能差异
- 通过统计指标量化了停顿与冲刷对 CPI 的影响

经验与建议：通过对比转发开关与分支冲刷的统计结果，能够直观理解“减少停顿”对性能的重要性，也为后续优化（如分支预测）提供方向。

附录：实验源代码

A.1 主要源码：main.py

```
import argparse
import shlex
from dataclasses import dataclass
from typing import Dict, List, Tuple


@dataclass
class Instruction:
    op: str
    args: Tuple
    text: str

    @staticmethod
    def nop() -> "Instruction":
        return Instruction("nop", tuple(), "nop")

    def is_nop(self) -> bool:
        return self.op == "nop"


@dataclass
class IFID:
    instr: Instruction
    pc: int


@dataclass
class IDEX:
    instr: Instruction
    pc: int
    rs: int
    rt: int
    rd: int
```

```
    imm: int
    rs_val: int
    rt_val: int


@dataclass
class EXMEM:
    instr: Instruction
    alu_result: int
    rt_val: int
    dest: int
    mem_read: bool
    mem_write: bool
    reg_write: bool
    mem_to_reg: bool
    branch_taken: bool
    branch_target: int


@dataclass
class MEMWB:
    instr: Instruction
    mem_data: int
    alu_result: int
    dest: int
    reg_write: bool
    mem_to_reg: bool


class MIPSPipelineSim:
    def __init__(self, forwarding: bool = True) -> None:
        self.forwarding = forwarding
        self.reset()

    def reset(self) -> None:
        self.regs = [0] * 32
        self.mem: Dict[int, int] = {}
        self.program: List[Instruction] = []
        self.text_labels: Dict[str, int] = {}
        self.data_labels: Dict[str, int] = {}
        self.halt = False
```

```
        self.pc = 0
        self.ifid = IFID(Instruction.nop(), 0)
        self.idex = IDEX(Instruction.nop(), 0, 0, 0, 0, 0, 0, 0)
        self.exmem = EXMEM(Instruction.nop(), 0, 0, 0, False, False,
           False, False, False, 0)
        self.memwb = MEMWB(Instruction.nop(), 0, 0, 0, False, False)
        self.cycles = 0
        self.completed = 0
        self.stalls = 0
        self.flushes = 0

    def load_program(self, text: str) -> None:
        self.reset()
        lines = text.splitlines()
        self._first_pass(lines)
        self.program = self._parse_instructions(lines)

    def _first_pass(self, lines: List[str]) -> None:
        section = "text"
        pc = 0
        data_addr = 0
        for raw in lines:
            line = raw.split("#", 1)[0].strip()
            if not line:
                continue
            if line.lower() == ".text":
                section = "text"
                continue
            if line.lower() == ".data":
                section = "data"
                continue

            label = None
            if ":" in line:
                label, rest = [p.strip() for p in line.split(":", 1)]
                line = rest
                if label:
                    if section == "text":
                        self.text_labels[label] = pc
                    else:
                        self.data_labels[label] = data_addr
```

```
            if not line:
                continue

        if section == "text":
            if not line.startswith("."):
                pc += 1
        else:
            if line.lower().startswith(".word"):
                values = self._split_args(line[5:])
                data_addr += 4 * len(values)
            else:
                raise ValueError(f"Unsupported data directive: {
                    line}")

def _parse_instructions(self, lines: List[str]) -> List[Instruction
    ]:
    program: List[Instruction] = []
    section = "text"
    data_addr = 0
    for raw in lines:
        line = raw.split("#", 1)[0].strip()
        if not line:
            continue
        if line.lower() == ".text":
            section = "text"
            continue
        if line.lower() == ".data":
            section = "data"
            continue
        if ":" in line:
            _, rest = [p.strip() for p in line.split(":", 1)]
            line = rest
            if not line:
                continue
        if section == "text":
            if line.startswith("."):
                continue
            instr = self._parse_instruction(line)
            program.append(instr)
        else:
            if line.lower().startswith(".word"):
```

```
                    values = self._split_args(line[5:])
                    for val in values:
                        word = self._parse_imm(val)
                        self._write_mem_word(data_addr, word)
                        data_addr += 4
                else:
                    raise ValueError(f"Unsupported data directive: {
                       line}")
        return program

    def _parse_instruction(self, line: str) -> Instruction:
        tokens = [t.strip() for t in line.replace(",", " ").split()]
        if not tokens:
            return Instruction.nop()
        op = tokens[0].lower()
        if op in ("addiu", "addi"):
            rt = self._parse_reg(tokens[1])
            rs = self._parse_reg(tokens[2])
            imm = self._parse_imm(tokens[3])
            return Instruction("addiu", (rt, rs, imm), line)
        if op == "add":
            rd = self._parse_reg(tokens[1])
            rs = self._parse_reg(tokens[2])
            rt = self._parse_reg(tokens[3])
            return Instruction(op, (rd, rs, rt), line)
        if op in ("lw", "sw"):
            rt = self._parse_reg(tokens[1])
            offset, rs = self._parse_mem(tokens[2])
            return Instruction(op, (rt, rs, offset), line)
        if op == "beq":
            rs = self._parse_reg(tokens[1])
            rt = self._parse_reg(tokens[2])
            label = tokens[3]
            if label not in self.text_labels:
                raise ValueError(f"Unknown label: {label}")
            target = self.text_labels[label]
            return Instruction(op, (rs, rt, target), line)
        if op == "beqz":
            rs = self._parse_reg(tokens[1])
            label = tokens[2]
            if label not in self.text_labels:
```

```
                raise ValueError(f"Unknown label: {label}")
            target = self.text_labels[label]
            return Instruction(op, (rs, target), line)
        if op == "teq":
            rs = self._parse_reg(tokens[1])
            rt = self._parse_reg(tokens[2])
            return Instruction(op, (rs, rt), line)
        if op == "nop":
            return Instruction.nop()
        raise ValueError(f"Unsupported op: {op}")

    def _parse_reg(self, token: str) -> int:
        token = token.strip().lower()
        if token.startswith("$"):
            token = token[1:]
        if token.startswith("r") and token[1:].isdigit():
            token = token[1:]
        if token.isdigit():
            reg = int(token)
            if 0 <= reg < 32:
                return reg
        raise ValueError(f"Invalid register: {token}")

    def _parse_mem(self, token: str) -> Tuple[int, int]:
        if "(" not in token or ")" not in token:
            raise ValueError(f"Invalid memory operand: {token}")
        offset_str, rest = token.split("(", 1)
        rs_str = rest.replace(")", "")
        offset = self._parse_imm(offset_str)
        rs = self._parse_reg(rs_str)
        return offset, rs

    def _parse_imm(self, token: str) -> int:
        token = token.strip()
        if token in self.data_labels:
            return self.data_labels[token]
        return int(token, 0)

    def _split_args(self, text: str) -> List[str]:
        return [t.strip() for t in text.replace(",", " ").split() if t.
            strip()]
```

```
    def _read_mem_word(self, addr: int) -> int:
        if addr % 4 != 0:
            raise ValueError(f"Unaligned memory access: {addr}")
        return self.mem.get(addr, 0)

    def _write_mem_word(self, addr: int, value: int) -> None:
        if addr % 4 != 0:
            raise ValueError(f"Unaligned memory access: {addr}")
        self.mem[addr] = value & 0xFFFFFFFF

    def _reg_read(self, reg: int) -> int:
        if reg == 0:
            return 0
        return self.regs[reg]

    def _reg_write(self, reg: int, value: int) -> None:
        if reg == 0:
            return
        self.regs[reg] = value & 0xFFFFFFFF

    def _instr_in_range(self, pc: int) -> bool:
        return 0 <= pc < len(self.program)

    def is_done(self) -> bool:
        if self.halt:
            return True
        if self.pc < len(self.program):
            return False
        return self.ifid.instr.is_nop() and self.idex.instr.is_nop()
           and self.exmem.instr.is_nop() and self.memwb.instr.is_nop()

    def step(self) -> None:
        if self.is_done():
            return

        self.cycles += 1

        old_ifid = self.ifid
        old_idex = self.idex
        old_exmem = self.exmem
```

```
        old_memwb = self.memwb

        # WB
        if not old_memwb.instr.is_nop():
            if old_memwb.reg_write:
                value = old_memwb.mem_data if old_memwb.mem_to_reg else
                    old_memwb.alu_result
                self._reg_write(old_memwb.dest, value)
            self.completed += 1

        # MEM
        mem_data = 0
        if not old_exmem.instr.is_nop():
            if old_exmem.mem_read:
                mem_data = self._read_mem_word(old_exmem.alu_result)
            if old_exmem.mem_write:
                self._write_mem_word(old_exmem.alu_result, old_exmem.
                    rt_val)
        new_memwb = MEMWB(
            old_exmem.instr,
            mem_data,
            old_exmem.alu_result,
            old_exmem.dest,
            old_exmem.reg_write,
            old_exmem.mem_to_reg,
        )

        # EX
        branch_taken = False
        branch_target = 0
        alu_result = 0
        dest = 0
        mem_read = False
        mem_write = False
        reg_write = False
        mem_to_reg = False
        rt_forward_val = old_idex.rt_val

        halt_now = False
        if not old_idex.instr.is_nop():
            rs_val, rt_val = self._apply_forwarding(old_idex)
```

```
            rt_forward_val = rt_val
            if old_idex.instr.op == "add":
                rd, _, _ = old_idex.instr.args
                alu_result = (rs_val + rt_val) & 0xFFFFFFFF
                dest = rd
                reg_write = True
            elif old_idex.instr.op == "addiu":
                rt, _, imm = old_idex.instr.args
                alu_result = (rs_val + imm) & 0xFFFFFFFF
                dest = rt
                reg_write = True
            elif old_idex.instr.op == "lw":
                rt, _, offset = old_idex.instr.args
                alu_result = (rs_val + offset) & 0xFFFFFFFF
                dest = rt
                mem_read = True
                reg_write = True
                mem_to_reg = True
            elif old_idex.instr.op == "sw":
                _, _, offset = old_idex.instr.args
                alu_result = (rs_val + offset) & 0xFFFFFFFF
                mem_write = True
            elif old_idex.instr.op == "beq":
                rs, rt, target = old_idex.instr.args
                if rs_val == rt_val:
                    branch_taken = True
                    branch_target = target
            elif old_idex.instr.op == "beqz":
                rs, target = old_idex.instr.args
                if rs_val == 0:
                    branch_taken = True
                    branch_target = target
            elif old_idex.instr.op == "teq":
                rs, rt = old_idex.instr.args
                if rs_val == rt_val:
                    halt_now = True
            elif old_idex.instr.op == "nop":
                pass
            else:
                raise ValueError(f"Unsupported op in EX: {old_idex.
                   instr.op}")
```

```
        new_exmem = EXMEM(
            old_idex.instr,
            alu_result,
            rt_forward_val,
            dest,
            mem_read,
            mem_write,
            reg_write,
            mem_to_reg,
            branch_taken,
            branch_target,
        )

        # ID
        stall = self._should_stall(old_ifid, old_idex, old_exmem,
           old_memwb)
        if stall:
            self.stalls += 1
            new_idex = IDEX(Instruction.nop(), 0, 0, 0, 0, 0, 0, 0)
        else:
            new_idex = self._decode(old_ifid)

        # IF
        flush = False
        fetch_pc = self.pc
        next_pc = self.pc
        if branch_taken:
            next_pc = branch_target
            flush = True
            self.flushes += 1
        elif halt_now:
            next_pc = len(self.program)
            flush = True
            self.halt = True
        elif not stall:
            next_pc = self.pc + 1

        new_ifid = old_ifid
        if not stall:
            instr = Instruction.nop()
```

```
            if self._instr_in_range(fetch_pc):
                instr = self.program[fetch_pc]
            new_ifid = IFID(instr, fetch_pc)

        if flush:
            new_ifid = IFID(Instruction.nop(), next_pc)
            new_idex = IDEX(Instruction.nop(), 0, 0, 0, 0, 0, 0, 0)

        self.pc = next_pc
        self.ifid = new_ifid
        self.idex = new_idex
        self.exmem = new_exmem
        self.memwb = new_memwb

    def _decode(self, ifid: IFID) -> IDEX:
        instr = ifid.instr
        if instr.is_nop():
            return IDEX(Instruction.nop(), 0, 0, 0, 0, 0, 0, 0)
        rs = rt = rd = imm = 0
        if instr.op == "add":
            rd, rs, rt = instr.args
        elif instr.op == "addiu":
            rt, rs, imm = instr.args
        elif instr.op in ("lw", "sw"):
            rt, rs, imm = instr.args
        elif instr.op == "beq":
            rs, rt, _ = instr.args
        elif instr.op == "beqz":
            rs, _ = instr.args
        elif instr.op == "teq":
            rs, rt = instr.args
        else:
            raise ValueError(f"Unsupported op in ID: {instr.op}")
        return IDEX(instr, ifid.pc, rs, rt, rd, imm, self._reg_read(rs)
            , self._reg_read(rt))

    def _apply_forwarding(self, idex: IDEX) -> Tuple[int, int]:
        rs_val = idex.rs_val
        rt_val = idex.rt_val
        if not self.forwarding:
            return rs_val, rt_val
```

```
        # EX/MEM forwarding (except load data)
        if self.exmem.reg_write and self.exmem.dest != 0 and not self.
            exmem.mem_to_reg:
            if self.exmem.dest == idex.rs:
                rs_val = self.exmem.alu_result
            if self.exmem.dest == idex.rt:
                rt_val = self.exmem.alu_result

        # MEM/WB forwarding
        if self.memwb.reg_write and self.memwb.dest != 0:
            wb_val = self.memwb.mem_data if self.memwb.mem_to_reg else
                self.memwb.alu_result
            if self.memwb.dest == idex.rs:
                rs_val = wb_val
            if self.memwb.dest == idex.rt:
                rt_val = wb_val

        return rs_val, rt_val

    def _should_stall(self, ifid: IFID, idex: IDEX, exmem: EXMEM, memwb
        : MEMWB) -> bool:
        instr = ifid.instr
        if instr.is_nop():
            return False

        rs, rt = self._get_source_regs(instr)

        # Load-use hazard (even with forwarding)
        if idex.instr.op == "lw":
            load_rt = idex.instr.args[0]
            if load_rt != 0 and (load_rt == rs or load_rt == rt):
                return True

        if self.forwarding:
            return False

        # No forwarding: stall on any RAW with results not yet written
        idex_dest = self._dest_reg(idex.instr)
        if self._writes_reg(idex.instr) and idex_dest != 0 and
            idex_dest in (rs, rt):
```

```
            return True
        if self._writes_reg(exmem.instr) and exmem.dest in (rs, rt):
            return True
        return False

    def _writes_reg(self, instr: Instruction) -> bool:
        if instr.is_nop():
            return False
        if instr.op in ("add", "addiu", "lw"):
            return True
        return False

    def _dest_reg(self, instr: Instruction) -> int:
        if instr.is_nop():
            return 0
        if instr.op == "add":
            rd, _, _ = instr.args
            return rd
        if instr.op == "addiu":
            rt, _, _ = instr.args
            return rt
        if instr.op == "lw":
            rt, _, _ = instr.args
            return rt
        return 0

    def _get_source_regs(self, instr: Instruction) -> Tuple[int, int]:
        if instr.op == "add":
            _, rs, rt = instr.args
            return rs, rt
        if instr.op == "addiu":
            _, rs, _ = instr.args
            return rs, 0
        if instr.op == "lw":
            _, rs, _ = instr.args
            return rs, 0
        if instr.op == "sw":
            rt, rs, _ = instr.args
            return rs, rt
        if instr.op == "beq":
            rs, rt, _ = instr.args
```

```
            return rs, rt
        if instr.op == "beqz":
            rs, _ = instr.args
            return rs, 0
        if instr.op == "teq":
            rs, rt = instr.args
            return rs, rt
        return 0, 0

    def dump_pipeline(self) -> str:
        def fmt(instr: Instruction) -> str:
            return instr.text

        return (
            f"IF/ID: {fmt(self.ifid.instr)}\n"
            f"ID/EX: {fmt(self.idex.instr)}\n"
            f"EX/MEM: {fmt(self.exmem.instr)}\n"
            f"MEM/WB: {fmt(self.memwb.instr)}"
        )

    def dump_regs(self) -> str:
        lines = []
        for i in range(0, 32, 4):
            chunk = " ".join([f"${j:02d}={self.regs[j]:08x}" for j in
                range(i, i + 4)])
            lines.append(chunk)
        return "\n".join(lines)

    def dump_stats(self) -> str:
        instrs = self.completed
        cpi = (self.cycles / instrs) if instrs else 0.0
        return (
            f"Cycles: {self.cycles}\n"
            f"Instructions: {instrs}\n"
            f"Stalls: {self.stalls}\n"
            f"Flushes: {self.flushes}\n"
            f"CPI: {cpi:.2f}"
        )

    def dump_source(self, start: int = 0, count: int = 0) -> str:
        if count <= 0:
```

```
            count = max(0, len(self.program) - start)
        end = min(len(self.program), start + count)
        lines = []
        for i in range(start, end):
            addr = i * 4
            marker = "->" if i == self.pc else "  "
            lines.append(f"{marker} {i:04d} (0x{addr:08x}): {self.
               program[i].text}")
        if not lines:
            return "<no instructions>"
        return "\n".join(lines)


class CLI:
    def __init__(self, sim: MIPSPipelineSim) -> None:
        self.sim = sim
        self.breakpoints: set[int] = set()

    def repl(self) -> None:
        print("MIPS 5-stage pipeline simulator. Type 'help' for
           commands.")
        while True:
            try:
                raw = input("mips> ").strip()
            except (EOFError, KeyboardInterrupt):
                print()
                break
            if not raw:
                continue
            parts = shlex.split(raw)
            cmd = parts[0].lower()
            args = parts[1:]
            try:
                if cmd in ("quit", "exit"):
                    break
                if cmd == "help":
                    self._help()
                elif cmd == "load":
                    self._cmd_load(args)
                elif cmd == "input":
                    self._cmd_input()
```

```
                elif cmd == "step":
                    self._cmd_step(args)
                elif cmd == "run":
                    self._cmd_run()
                elif cmd == "runto":
                    self._cmd_runto(args)
                elif cmd == "break":
                    self._cmd_break(args)
                elif cmd == "breaks":
                    self._cmd_breaks()
                elif cmd == "clear":
                    self._cmd_clear(args)
                elif cmd == "regs":
                    print(self.sim.dump_regs())
                elif cmd == "list":
                    self._cmd_list(args)
                elif cmd == "pipe":
                    print(self.sim.dump_pipeline())
                elif cmd == "mem":
                    self._cmd_mem(args)
                elif cmd == "memset":
                    self._cmd_memset(args)
                elif cmd == "stats":
                    print(self.sim.dump_stats())
                elif cmd == "forwarding":
                    self._cmd_forwarding(args)
                elif cmd == "reset":
                    self.sim.reset()
                    print("Simulator reset.")
                else:
                    print(f"Unknown command: {cmd}")
            except Exception as exc:
                print(f"Error: {exc}")

    def _help(self) -> None:
        print(
            "Commands:\n"
            "  load <file>           Load program from file\n"
            "  input                 Enter program from stdin, finish
               with .end\n"
            "  step [n]              Execute 1 (or n) cycles\n"
```

```
            "  run                    Run until breakpoint or program
               end\n"
            "  runto <pc|label>       Run until pc reaches target\n"
            "  break <pc|label>       Set breakpoint\n"
            "  breaks                 List breakpoints\n"
            "  clear <pc|all>         Clear breakpoint\n"
            "  regs                   Show registers\n"
            "  list [start] [count]   Show source with addresses\n"
            "  mem <addr> <count>     Dump memory words\n"
            "  memset <addr> <value>  Set one memory word\n"
            "  pipe                   Show pipeline registers\n"
            "  stats                  Show performance stats\n"
            "  forwarding on|off      Toggle data forwarding\n"
            "  reset                  Reset pipeline and state\n"
            "  quit                   Exit\n"
        )

    def _cmd_load(self, args: List[str]) -> None:
        if len(args) != 1:
            raise ValueError("Usage: load <file>")
        with open(args[0], "r", encoding="utf-8") as f:
            text = f.read()
        self.sim.load_program(text)
        print(f"Loaded {len(self.sim.program)} instructions.")

    def _cmd_input(self) -> None:
        print("Enter program, end with .end")
        lines = []
        while True:
            line = input()
            if line.strip() == ".end":
                break
            lines.append(line)
        self.sim.load_program("\n".join(lines))
        print(f"Loaded {len(self.sim.program)} instructions.")

    def _cmd_step(self, args: List[str]) -> None:
        count = int(args[0]) if args else 1
        for _ in range(count):
            if self.sim.is_done():
                print("Program complete.")
```

```
                break
            self.sim.step()
        print(f"PC={self.sim.pc}, cycles={self.sim.cycles}")
        print(self.sim.dump_source(self.sim.pc, 5))

    def _cmd_run(self) -> None:
        while not self.sim.is_done():
            if self.sim.pc in self.breakpoints:
                print(f"Hit breakpoint at PC={self.sim.pc}")
                break
            self.sim.step()
        if self.sim.is_done():
            print("Program complete.")

    def _cmd_runto(self, args: List[str]) -> None:
        if len(args) != 1:
            raise ValueError("Usage: runto <pc|label>")
        target = self._resolve_target(args[0])
        while not self.sim.is_done():
            if self.sim.pc == target:
                print(f"Reached PC={self.sim.pc}")
                break
            self.sim.step()

    def _cmd_break(self, args: List[str]) -> None:
        if len(args) != 1:
            raise ValueError("Usage: break <pc|label>")
        target = self._resolve_target(args[0])
        self.breakpoints.add(target)
        print(f"Breakpoint set at PC={target}")

    def _cmd_breaks(self) -> None:
        if not self.breakpoints:
            print("No breakpoints.")
            return
        print("Breakpoints:")
        for bp in sorted(self.breakpoints):
            print(f"  PC={bp}")

    def _cmd_clear(self, args: List[str]) -> None:
        if len(args) != 1:
```

```
            raise ValueError("Usage: clear <pc|all>")
        if args[0] == "all":
            self.breakpoints.clear()
            print("All breakpoints cleared.")
            return
        target = self._resolve_target(args[0])
        self.breakpoints.discard(target)
        print(f"Breakpoint cleared at PC={target}")

    def _cmd_mem(self, args: List[str]) -> None:
        if len(args) != 2:
            raise ValueError("Usage: mem <addr> <count>")
        addr = int(args[0], 0)
        count = int(args[1], 0)
        for i in range(count):
            a = addr + i * 4
            val = self.sim.mem.get(a, 0)
            print(f"0x{a:08x}: 0x{val:08x}")

    def _cmd_list(self, args: List[str]) -> None:
        if len(args) > 2:
            raise ValueError("Usage: list [start] [count]")
        start = int(args[0], 0) if len(args) >= 1 else 0
        count = int(args[1], 0) if len(args) == 2 else 0
        print(self.sim.dump_source(start, count))

    def _cmd_memset(self, args: List[str]) -> None:
        if len(args) != 2:
            raise ValueError("Usage: memset <addr> <value>")
        addr = int(args[0], 0)
        value = int(args[1], 0)
        self.sim._write_mem_word(addr, value)
        print(f"0x{addr:08x}: 0x{value:08x}")

    def _cmd_forwarding(self, args: List[str]) -> None:
        if len(args) != 1:
            raise ValueError("Usage: forwarding on|off")
        if args[0] == "on":
            self.sim.forwarding = True
        elif args[0] == "off":
            self.sim.forwarding = False
```

```
        else:
            raise ValueError("Usage: forwarding on|off")
        print(f"Forwarding {'on' if self.sim.forwarding else 'off'}")

    def _resolve_target(self, token: str) -> int:
        if token.isdigit():
            return int(token)
        if token in self.sim.text_labels:
            return self.sim.text_labels[token]
        raise ValueError(f"Unknown target: {token}")


def main() -> None:
    parser = argparse.ArgumentParser(description="MIPS 5-stage pipeline
        simulator")
    parser.add_argument("-f", "--file", help="Program file to load")
    parser.add_argument("--forwarding", choices=["on", "off"], default=
       "on")
    args = parser.parse_args()

    sim = MIPSPipelineSim(forwarding=(args.forwarding == "on"))
    cli = CLI(sim)

    if args.file:
        with open(args.file, "r", encoding="utf-8") as f:
            sim.load_program(f.read())
        print(f"Loaded {len(sim.program)} instructions from {args.file}
           ")

    cli.repl()


if __name__ == "__main__":
    main()
```

A.2 运行命令

```
# 无冒险示例
python main.py -f examples/no_hazard.s
```

```
# RAW冒险示例
python main.py -f examples/raw_hazard.s

# 分支控制冒险示例
python main.py -f examples/branch_jump.s
```